

3位 解法

Rank	3
Team	W & A
Author	Arunodhayan
Topic ID	679236
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679236>

Retrieved with Kaggle CLI on 2026-07-07.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

主催者に感謝する。高度にcustomizeした**nnU-Net v2**を基盤とし、train/inferenceで非対称なpatch size、2-stage Cascade 3D、custom TTA、3D Hessian ridge detection post-processingを組み合わせた。

1. 全体architecture

- **Stage 1 (3D Fullres):** full-resolution 3D modelで初期予測し、複数modelのprobability mapをweighted ensemble。
- **Stage 2 (3D Cascade Fullres):** Stage 1予測をspatial priorとしてcascade modelへ入力しrefine。
- **Post-processing:** Distance Transformと3D Hessianで微細構造抽出とtopology最適化。

2. Dataset戦略

最終modelはK-Foldでholdoutせず**全train data**で訓練した。external/unlabeled dataのpseudo-labelも試したが有意な改善がなく不採用。nnU-Net default augmentationに加え全axis flip (0,1,2)を強制し、rotation probabilityを0.4~0.8へ上げた。

3. Modelと訓練

- **Asymmetric Patch Size:** M-size architectureを使い、CZIIで有効だったsize decouplingを採用。trainは128、inferenceは192。train時のbatch size・iteration効率を保ち、推論時のreceptive fieldを広げて大域一貫性を改善。
- **Topology-aware loss:** default lossへ**clDice Loss**を追加し、細長い・mesh状構造のskeleton connectivityを最適化。
- **Optimizer:** schedulerを廃し**RAdamScheduleFree**へ変更。収束を滑らかにし訓練効率を改善。
- **Cascade training:** Lowresを先に訓練し、それを基にCascadeを訓練するcoarse-to-fine pathでprevious-stage predictionを正しいspatial priorとして学習。

4. 推論とTTA

`predict_from_raw_data.py`を書き換えた。192 patchで、Fullres出力をCascadeのfeature priorへ直接使用。通常のmirrorに加えAffine Grid Sampleのin-plane rotation TTA (±15°)を導入した。trainでは±30° random rotationだったが、評価上TTAは±15°が最良。train rotationをさらに広げると悪化した。multi-processをGPUへ固定するparallel scriptで `.npz` probability mapを抽出し、iterationや設定の異なるcheckpointをequal/weighted fusionした。

5. Topology post-processing

1. **Inverse EDT Dilation:** Euclidean Distance Transformのinverse mappingでbinary結果をnon-linear dilation。

2. **3D Hessian Ridge Detection:** targetを空間ridgeとみなし、Gaussian smoothing後の3D Hessian固有値から feature spaceを構成。tubular構造を抽出し非構造background noiseを強く除外。
3. **Pruning:** 現在のZ sliceだけpositiveで上下sliceがnegativeのvoxelをfalse positiveとして除去。2D closingで slice境界を平滑化し、XY planeのanisotropic 3D closingで断裂を修復。最後に小さい3D noiseを除去。

post-processingの多くは主催者が共有した[Discussion](#)と[code](#)を参考にした。

Best Private submissionは lowres を8000 epochs、その上の cascade をさらに8000 epochs訓練した [configuration](#)だった。

補足Q&A

ArjunB: post-processingによるPublic/Private改善は？

Arunodhayan: 約0.02〜0.03だと思う。

Sean Johnson_SP: ridge detectionとEDT dilationはVesuvius Python libraryのcleanup filter由来ですか？

wpl: はい。最終post-processing codeの大部分は共有された手法に由来し、単純な変更を加えた。

Tom: Hessian処理は長いがruntimeは？

wpl: 約1時間と見積もる。

Maapu: 8000 epochsを決断できたtraining speedは？

Arunodhayan: 2×H100で、3d lowresの8000 epochsが14時間、3d cascadeが24時間。

CPMP: Hessian ridge detectionの実装は？

Arunodhayan: 上記のsubmission notebookに実装を添付した。